Matthew Bernardino
Professor Anshul
ECPE 136
17 April 2026

Lab Assignment 9 - 64-Byte Synchronous Memory

Objective:

- The objective of this lab is to create a 64 byte synchronous memory storage device in the AMD Vivado to write data into a desired address. We should also be able to read from the address where the data was stored and see that the value written into the address is returned when read. This project uses a clock to simulate a real design where changes are made when the clock is high. We also need to incorporate a reset to which all the addresses in memory will be cleared to zero.

Procedures:

1.0 - Create a Project/Create File

- In AMD Vivado we need to create a project with the title for the lab assignment. This is where all the values would store the files related to the project. Next it asks if we want to create a file in which we will do and name "**sync_mem_64b**" in which we will be coding in system verilog. In this file we can create the top module that will handle all the variables for inputs and outputs.

2.0 - Create a Testbench File / Test

- When completing the top module file we will create a testbench that allows us to input values to test that the output is correct on a waveform. Within this testbench "**sync_mem_64tb.sv**" we will set up our same variables and instantiate the same variables to the top modules so that the input values in the test bench go into the same variables as the top module. Next, we will test different variables and make note of what the output should be for when simulating the test bench. Run the simulation with the inputs implemented and see that the output values on the waveform reflect correctly.

Design description:

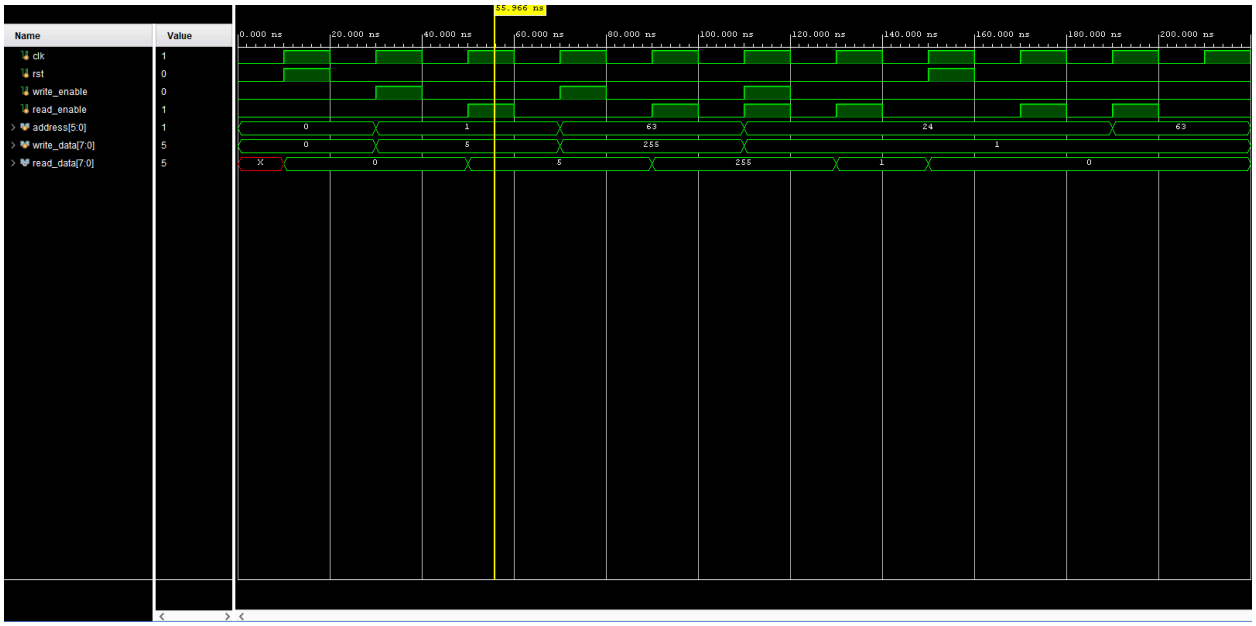
```
…
// when write_enable = 1, store write_data into memory address
      // prioritize write enable since it is the next first else if it will get priortity when checking
      else if (write_enable == 1) begin
         mem[address] <= write_data;  // memory address will be updated to write_data
      end

      // want to copy the memory value in the output when 'read_enable' = 1
      else if (read_enable == 1) begin
          read_data <= mem[address]; // read_data takes memory address value
      end
…
```

sync_mem_64b.sv snippet

- I chose to make write enable the priority within this lab. Using if statements I made the initial reset as the priority if it ever is set to 1 to clear the memory. The next step was to use else if statements in which I made it so that the write enable statement came before read enable. This means that as the code executes it will execute the write enable statement first following that and ignoring the read enable statement. Within the test bench I specifically turned both on and the value I wanted to be written into an address was written in and the next read I did showed the value in memory.

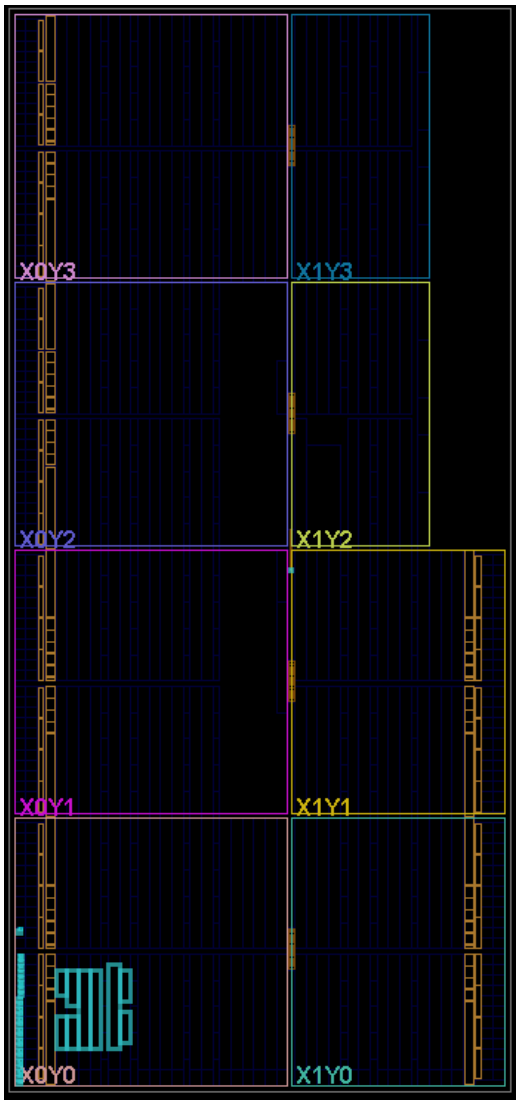
Results & Observations:



64-Byte Synchronous Memory Waveform

Observation:

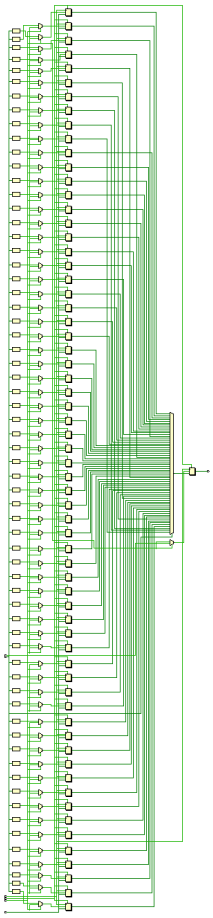
- Looking at the waveform, the test bench worked really well. I set the clock to have intervals of 20 ns. This means that each high and low state is 10ns wide. The goal is to see that values update on the clock high and do not on clk low. When looking at the waveform we start with a reset to verify all address values are zero before updating. I began with a write_enable on to update the address 1 with the value of 5. The next statement turns off write_enable and turns on read_enable to verify that address 1 has the value 5 stored on it. This is proven when looking at the output waveform result. In my testing for priority of write_enable, I turned both write_enable and read_enable on. I wanted the address 24 to update with the value 1. Since it began on clock low it did not begin until clock high. On the next read_enable the value 1 was stored and read from address 24 proving my priority works and not just reading the previous value stored to address 24 which would have been zero.



64-Byte Synchronous Memory Design implementation

Observation:

- When looking at the design implementation, we can see that there is only one quadrant where there are new modules representing the design of the memory storage device. There should be 64 row addresses that are 64 bytes wide. Since the simulate re-uses the same variables and circuit design it is linear and the same as more quadrants are shown.



64-Byte Synchronous Memory RTL Design

Observation:

- I believe in this image we can see the 64 storage locations within this memory design. Then they all go through a mux that would determine based on the logic if a value is stored or read.

Conclusion:

- The goal of this lab was to re-visit AMD Vivado and code in system verilog a 64 byte synchronous memory storage device. The goal was to implement a clock, reset, write_enable, read_enable, address, and write_data. Using these variables we create a storage device that has 64 available addresses that are 64 bit wide. When a write_enable is on then we can write to memory a value into an address. When read_enable is on then we can read the value from an address. When both are active depending on the priority design then write_enable or read_enable will be prioritized. I prioritized write_enable so if both are on read_enable will not be followed. Overall, we re-visit the AMD Vivado software to learn about memory.

Appendix

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 04/14/2026 04:09:43 PM
// Design Name:
// Module Name: sync_mem_64b
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
//
//////////////////////////////////////////////////////////////////////////////////
// Matthew Bernardino
// Professor Anshul
// ECPE 192
// 17 April 2026
// Description: to create a synchronous memory byte addressable storage of 64 bytes

module sync_mem_64b(
    input logic clk,
    input logic rst,
    input logic read_enable,
    input logic write_enable,
    input logic [5:0] address,     // 6 bits
    input logic [7:0] write_data,  // 8 bits
    output logic [7:0] read_data    // 8 bits
    );

    // create memory variable
    logic [7:0] mem [0:63];     // 8 bit memory for 64 rows

    always_ff @(posedge clk) begin // since we use clock we want positive edge to be when
executing operations
    // use <= for synchronous to update on next high clock
```

```
    if (rst == 1) begin // make the priotrity the reset for all stored memory
        for (integer i=0; i<64; i++) begin //loop for 64 8 bit address // make variable i
            mem[i] <= 8'b0; // set all to 0
        end

        read_data <= 8'b0; //clear read_data so set to 0 aswell

    end

    // when write_enable = 1, store write_data into memory address
    // prioritize write enable since it is the next first else if it will get priortity when checking
    else if (write_enable == 1) begin
        mem[address] <= write_data;  // memory address will be updated to write_data
    end

    // want to copy the memory value in the output when 'read_enable' = 1
    else if (read_enable == 1) begin
         read_data <= mem[address]; // read_data takes memory address value
    end

  end

endmodule
```

sync_mem_64b.sv

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////
// Company:
// Engineer:
//
// Create Date: 04/14/2026 04:11:05 PM
// Design Name:
// Module Name: sync_mem_64tb
// Project Name:
// Target Devices:
// Tool Versions:
// Description:
//
// Dependencies:
//
// Revision:
// Revision 0.01 - File Created
// Additional Comments:
```

```
//
//////////////////////////////////////////////////////////////////////////////////
// Matthew Bernardino
// Professor Anshul
// ECPE 192
// 17 April 2026
// Description: to create a synchronous memory byte addressable storage of 64 bytes test bench

// declare our variables same as main.sv
module sync_mem_64tb();
    // input variabels
    reg clk;
    reg rst;
    reg write_enable;
    reg read_enable;
    reg [5:0] address; // can only have 2^6 = 64 max address
    reg [7:0] write_data;

    // output variable
    wire [7:0] read_data;

    // instantiate with the same variables as the main.sv
    sync_mem_64b inst(.clk(clk), .rst(rst), .read_enable(read_enable),
.write_enable(write_enable),
    .address(address), .write_data(write_data), .read_data(read_data));

    initial begin
        clk = 0;
        forever #10 clk = ~clk; // 20 ns total, 10ns high and 10ns low
    end

    initial begin
        // set all variables to 0
        //clk = 0;
        rst = 0;
        read_enable = 0;
        write_enable = 0;
        address = 6'b0;  // 6 bit value
        write_data = 8'b0; // 8 bit value
        #10; // delay

        // memory reset to show all are 0
        rst = 1;
        #10;
        rst = 0; // reset no longer on so values can populate in memory
        #10;
```

```
// write decimal 5 to address 1
write_enable = 1;   // on
read_enable = 0;    // off
address = 6'd1;    // address 1 will recieve 5 number
write_data = 8'd5; // 5 to input into address 1
#10;
write_enable = 0; // turn back off
#10;

// read decimal 5 from address 1
read_enable = 1; // on
address = 6'd1;    // address 1 will be read = 5
#10;
read_enable = 0; // turn back off
#10;

// write decimal 255 to address 63 // edge case
write_enable = 1;   // on
read_enable = 0;    // off
address = 6'd63;    // address 63 will recieve 255 number
write_data = 8'd255; // 255 to input into address 63
#10;
write_enable = 0; // turn back off
#10;

// read decimal 255 from address 63
read_enable = 1; // on
address = 6'd63;    // address 63 will be read = 255
#10;
read_enable = 0; // turn backs off
#10;

// Conflict Case were both write and read enable are set to 1
// write_enable should be prioritized
// write decimal 1 to address 24 // edge case
write_enable = 1;   // on
read_enable = 1;    // on
address = 6'd24;    // address 24 will recieve 1 number
write_data = 8'd1; // 1 to input into address 24
#10;
write_enable = 0; // turn back off
read_enable = 0;  // turn back off
#10;

// read decimal 1 from address 24
```

```
        read_enable = 1; // on
        address = 6'd24;    // address 24 will be read = 1
        #10;
        read_enable = 0; // turn backs off
        #10;

        // memory reset to show all are 0
        rst = 1;
        #10;
        rst = 0; // reset no longer on so values can populate in memory
        #10;

        // read from address 24
        read_enable = 1; // on
        address = 6'd24;    // address 24 will be read = 0 due to reset rst = 1
        #10;
        read_enable = 0; // turn backs off
        #10;

        // read from address 63
        read_enable = 1; // on
        address = 6'd63;    // address 63 will be read = 0 due to reset rst = 1
        #10;
        read_enable = 0; // turn backs off
        #10;

        #10 $finish;

    end

    // Monitor output
    always@(clk, rst, write_enable, read_enable, address, write_data)
        $monitor("Time=%0t ns, clk=%0b, rst=%0b, write_enable=%0b, read_enable=%0b,
address=%0d, write_data=%0d", $time, clk, rst, write_enable, read_enable, address,
write_data);

endmodule
```

sync_mem_64tb.sv